TP1 : Résolution de systèmes d'équations linéaires

07/10/2026

ILIN Nikita, KOLODNYTSKA Viktoriia

1 Introduction

Dans ce TP, on résout des systèmes linéaires Ax=b par une méthode directe-la méthode de Gauss, puis par deux méthodes itératives, Jacobi et Gauss-Seidel.

2 Rappel rapide des méthodes

2.1 Méthode de Gauss

La méthode de Gauss est une méthode directe. Elle consiste d'abord à transformer le système en un système triangulaire supérieur en éliminant progressivement les coefficients situés sous la diagonale.

Pour éliminer un coefficient, on utilise le coefficient :

$$k = \frac{a_{ic}}{a_{cc}}$$

puis on effectue l'opération :

$$L_i = L_i - kL_c$$

Une fois la matrice triangulaire obtenue, on calcule les valeurs de la solution en partant de la dernière équation. Ce qu'on appelle la substitution arrière.

2.2 Méthode de Jacobi

La méthode de Jacobi est une méthode itérative. On commence avec une solution initiale nulle.

À chaque itération, chaque nouvelle valeur est calculée à partir des valeurs de l'itération précédente :

$$x_i^{(k+1)} = \frac{b_i - \sum_{j \neq i} a_{ij} x_j^{(k)}}{a_{ii}}$$

Dans le programme, deux vecteurs sont donc utilisés - le vecteur de l'itération précédente et le nouveau vecteur.

2.3 Méthode de Gauss-Seidel

La méthode de Gauss-Seidel est proche de la méthode de Jacobi, mais les nouvelles valeurs calculées sont utilisées immédiatement pendant la même itération.

Il n'y adonc pas besoin d'utiliser un deuxième vecteur pour stocker toute la nouvelle solution avant de mettre à jour l'ancienne.

2.4 Critère d'arrêt

Dans les deux méthodes itératives, la solution initiale choisie est le vecteur nul.

Les itérations s'arrêtent lorsque l'erreur devient inférieure à une valeur donnée ou lorsque le nombre maximal d'itérations est atteint.

Comme dans ce TP la solution exacte est :

$$x_i = 1$$

l'erreur utilisée dans le programme est :

$$p_k = \max_i |1 - x_i^{(k)}|$$

3 Présentation des programmes commentés

3.1 Méthode de Gauss

Le premier programme implémente la méthode de Gauss.

Le vecteur b est construit de manière à avoir comme solution exacte un vecteur composé uniquement de 1. Pour cela, chaque valeur de b est obtenue en faisant la somme des éléments de la ligne correspondante de la matrice A.

Le programme effectue ensuite l'élimination de Gauss afin de transformer la matrice en matrice triangulaire supérieure puis calcule la solution par substitution arrière.

Le programme est testé sur les quatre matrices A_1, A_2, A_3 et A_4.

Listing 1 – Programme méthode de Gauss

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  void afficher(double **tab,int n,int m){
5      int i,j;
6
7      for(i=0;i<n;i++){
8          for(j=0;j<m;j++){                          // affiche une
               matrice de n lignes et m colonnes
9              printf("%lf ",tab[i][j]);
10         }
11         printf("\n");
12     }
13 }
14
15
16
```

```
17 // résout le système Ax=b par la méthode de Gauss
18 // le vecteur b est calculé de sorte que la solution exacte soit x
       =(1,..,1)
19 double **gauss(double **tab,int n){
20     int i,j,c;
21     double k;
22     // Création du second membre b
23     double *b=malloc(n*sizeof(double));
24     if(b==NULL){
25         return NULL;
26     }
27
28
29     for(i=0;i<n;i++){
30         double somme=0;
31         for(j=0;j<n;j++){                    // calcul de b=A*x
32             somme+=tab[i][j];                // chaque élément de b
                   est donc la somme d'une ligne de A
33         }
34         b[i]=somme;
35     }
36
37     // transformation de la matrice en matrice triangulaire supé
          rieure
38     for(c=0;c<n-1;c++){
39         for(i=c+1;i<n;i++){
40             k=tab[i][c]/tab[c][c]; // k est le coefficient utilisé
                   pour éliminer l'élément tab[i][c]
41             for(j=c;j<n;j++){
42                 tab[i][j]-=k*tab[c][j];
43             }
44             b[i]-=k*b[c];   // même opération effectuée sur le
                   second membre
45         }
46     }
47
48      // création du vecteur solution x
49     double **x=malloc(n*sizeof(double *));
50     if(x==NULL){
51         free(b);
52         return NULL;
53     }
54
55     for(i=0;i<n;i++){
56         x[i]=malloc(sizeof(double));
57     }
58
59
60     // Substitution arrière
61     // On commence par la dernière équation
62     for(i=n-1;i>=0;i--){
```

```
63          double somme=0;
64          for(j=i+1;j<n;j++){
65              somme+=tab[i][j]*x[j][0];
66          }
67          x[i][0]=(b[i]-somme)/tab[i][i];
68      }
69
70      free(b);
71
72      return x;
73  }
74
75  int main(){
76      int n=3;
77      int i;
78
79      double *A1[]={ (double[]){3,0,4},(double[]){7,4,2},(double[])
            {-1,1,2}};
80      double *A2[]={(double[]){-3,3,-6},(double[]){-4,7,8},(double[])
            {5,7,-9}};
81      double *A3[]={(double[]){4,1,1},(double[]){2,-9,0},(double[])
            {0,-8,6}};
82      double *A4[]={(double[]){7,6,9},(double[]){4,5,-4},(double[])
            {-7,-3,8}};
83
84      // résolution des systèmes
85      double **X1=gauss(A1,n);
86      double **X2=gauss(A2,n);
87      double **X3=gauss(A3,n);
88      double **X4=gauss(A4,n);
89      // affichage des solutions
90      afficher(X1,n,1);
91      afficher(X2,n,1);
92      afficher(X3,n,1);
93      afficher(X4,n,1);
94
95      // libération des vecteurs solutions
96      for(i=0;i<n;i++){
97          free(X1[i]);
98          free(X2[i]);
99          free(X3[i]);
100         free(X4[i]);
101     }
102     free(X1);
103     free(X2);
104     free(X3);
105     free(X4);
106
107     return 0;
108 }
```

3.2 Méthodes de Jacobi et Gauss-Seidel

Le deuxième programme contient les deux méthodes itératives.

Pour les deux méthodes le vecteur initial est nul et le vecteur b est construit de la même manière que dans le programme précédent.

Dans la méthode de Jacobi les nouvelles valeurs sont stockées dans un deuxième vecteur avant de remplacer les anciennes valeurs.

Dans la méthode de Gauss-Seidel chaque nouvelle valeur est directement enregistrée dans le vecteur solution et peut donc être utilisée pour calculer les valeurs suivantes.

À chaque itération l'erreur est calculée afin de vérifier la convergence de la méthode. Le programme s'arrête lorsque l'erreur devient inférieure à 10^{-6} ou lorsque 1000 itérations ont été réalisées.

Les méthodes sont testées sur A_1, A_2, A_3 et A_4, ainsi que sur les matrices A_5 et A_6 pour les tailles $n = 3$, 6, 8 et 10.

Listing 2 – Programme des méthodes de Jacobi et Gauss-Seidel

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

double *jacobi(double **tab,int n,double epsilon,int maxIter){
    int i,j,k=0;
    double somme;
    double err=1;

    double *b=malloc(n*sizeof(double));
    double *x=malloc(n*sizeof(double));
    double *x2=malloc(n*sizeof(double));

    // calcul de b pour avoir x=(1,...,1)
    for(i=0;i<n;i++){
        somme=0;
        for(j=0;j<n;j++){
            somme+=tab[i][j];
        }
        b[i]=somme;
    }

    // solution initiale nulle
    for(i=0;i<n;i++){
        x[i]=0;
    }

    while(err>epsilon&&k<maxIter){
        // calcul de la nouvelle approximation avec les valeurs de
           l'iteration precedente
        for(i=0;i<n;i++){
            somme=0;
            for(j=0;j<n;j++){
                if(j!=i)
                    somme+=tab[i][j]*x[j];
```

```
35              }
36              x2[i]=(b[i]-somme)/tab[i][i];
37          }
38          for(i=0;i<n;i++){
39              x[i]=x2[i];      //mise à jour de la solution apres
                    avoir calculé toutes les nouvelles valeurs
40          }
41
42          // erreur relative
43          err=0;
44
45          for(i=0;i<n;i++){
46              if(fabs(1-x[i])>err)
47                  err=fabs(1-x[i]);
48          }
49          k++;
50          printf("%d %e\n",k,err);
51      }
52
53      printf("Nombre d'iterations Jacobi: %d\n",k);
54      printf("erreur finale: %e\n",err);
55
56      free(b);
57      free(x2);
58      return x;
59  }
60
61
62  double *gaussSeidel(double **tab,int n,double epsilon,int maxIter){
63      int i,j,k=0;
64      double somme;
65      double err=1;
66      double *b=malloc(n*sizeof(double));
67      double *x=malloc(n*sizeof(double));
68
69      // calcul de b pour avoir x=(1,..,1)
70      for(i=0;i<n;i++){
71          somme=0;
72          for(j=0;j<n;j++){
73              somme+=tab[i][j];
74          }
75          b[i]=somme;
76      }
77
78      // solution initiale nulle
79      for(i=0;i<n;i++){
80          x[i]=0;
81      }
82
83      while(err>epsilon&&k<maxIter){
84          //mise à jour directe de x
```

```
85          // utilisation immediate des nouvelles valeurs
86          for(i=0;i<n;i++){
87              somme=0;
88
89              for(j=0;j<n;j++){
90                  if(j!=i){
91                      somme+=tab[i][j]*x[j];
92                  }
93              }
94              x[i]=(b[i]-somme)/tab[i][i];
95          }
96
97          // erreur relative
98          err=0;
99
100         for(i=0;i<n;i++){
101             if(fabs(1-x[i])>err){
102                 err=fabs(1-x[i]);
103             }
104         }
105         k++;
106         printf("%d %e\n",k,err);
107     }
108
109     printf("Nombre d'iterations Gauss-Seidel: %d\n",k);
110     printf("erreur finale: %e\n",err);
111
112     free(b);
113     return x;
114 }
115
116
117 int main(){
118     int i,j,m,t;
119     int n=3;
120     double epsilon=0.000001;
121     int maxIter=1000;
122
123     double valeurs[4][3][3]={
124         {{3,0,4},{7,4,2},{-1,1,2}},
125         {{-3,3,-6},{-4,7,8},{5,7,-9}},
126         {{4,1,1},{2,-9,0},{0,-8,6}},
127         {{7,6,9},{4,5,-4},{-7,-3,8}}
128     };
129
130     // tests de A1 à A4
131     for(m=0;m<4;m++){
132         double **A=malloc(n*sizeof(double *));
133         for(i=0;i<n;i++){
134             A[i]=malloc(n*sizeof(double));
135             for(j=0;j<n;j++){
```

```
                A[i][j]=valeurs[m][i][j];
            }
        }

        printf("\n=======================\n");
        printf("Matrice A%d\n",m+1);
        printf("=======================\n");

        printf("\nJacobi:\n");

        double *xJ=jacobi(A,n,epsilon,maxIter);

        printf("Solution:\n");
        for(i=0;i<n;i++){
            printf("x%d=%lf\n",i+1,xJ[i]);
        }

        printf("\nGauss-Seidel:\n");
        double *xGS=gaussSeidel(A,n,epsilon,maxIter);
        printf("Solution:\n");

        for(i=0;i<n;i++){
            printf("x%d=%lf\n",i+1,xGS[i]);
        }
        free(xJ);
        free(xGS);

        for(i=0;i<n;i++){
            free(A[i]);
        }
        free(A);
    }


    int tailles[4]={3,6,8,10};

    // tests de A5
    for(t=0;t<4;t++){
        n=tailles[t];
        double **A5=malloc(n*sizeof(double *));
        for(i=0;i<n;i++){
            A5[i]=malloc(n*sizeof(double));
            for(j=0;j<n;j++){
                if(i==j)
                    A5[i][j]=1;
                else if(i==0)
                    A5[i][j]=pow(2,-j);
                else if(j==0)
                    A5[i][j]=pow(2,-i);
                else
                    A5[i][j]=0;
```

```
            }
        }

        printf("\n=======================\n");
        printf("Matrice A5, n=%d\n",n);
        printf("=======================\n");
        printf("\nJacobi:\n");

        double *xJ=jacobi(A5,n,epsilon,maxIter);
        printf("Solution:\n");

        for(i=0;i<n;i++){
            printf("x%d=%lf\n",i+1,xJ[i]);
        }

        printf("\nGauss-Seidel:\n");
        double *xGS=gaussSeidel(A5,n,epsilon,maxIter);
        printf("Solution:\n");

        for(i=0;i<n;i++){
            printf("x%d=%lf\n",i+1,xGS[i]);
        }
        free(xJ);
        free(xGS);

        for(i=0;i<n;i++){
            free(A5[i]);
        }
        free(A5);
    }


    // tests de A6
    for(t=0;t<4;t++){
        n=tailles[t];
        double **A6=malloc(n*sizeof(double *));
        for(i=0;i<n;i++){
            A6[i]=malloc(n*sizeof(double));
            for(j=0;j<n;j++){
                if(i==j)
                    A6[i][j]=3;
                else if(j==i+1)
                    A6[i][j]=-1;
                else if(j==i-1)
                    A6[i][j]=-2;
                else
                    A6[i][j]=0;
            }
        }

        printf("\n=======================\n");
```

```
238         printf("Matrice A6, n=%d\n",n);
239         printf("========================\n");
240         printf("\nJacobi:\n");
241 
242         double *xJ=jacobi(A6,n,epsilon,maxIter);
243 
244         printf("Solution:\n");
245         for(i=0;i<n;i++){
246             printf("x%d=%lf\n",i+1,xJ[i]);
247         }
248 
249         printf("\nGauss-Seidel :\n");
250         double *xGS=gaussSeidel(A6,n,epsilon,maxIter);
251         printf("Solution :\n");
252 
253         for(i=0;i<n;i++){
254             printf("x%d=%lf\n",i+1,xGS[i]);
255         }
256         free(xJ);
257         free(xGS);
258 
259         for(i=0;i<n;i++){
260             free(A6[i]);
261         }
262 
263         free(A6);
264     }
265 
266     return 0;
267 }
```

4 Résultats et commentaires

Cette partie présentera les résultats obtenus avec les différentes matrices.

On comparera le nombre d'itérations nécessaire pour les méthodes de Jacobi et de Gauss-Seidel ainsi que l'évolution de l'erreur.

4.1 Résultats de la méthode de Gauss

La méthode de Gauss donne la solution attendue pour les quatre matrices :

Matrice	x_1	x_2	x_3
A_1	1.000000	1.000000	1.000000
A_2	1.000000	1.000000	1.000000
A_3	1.000000	1.000000	1.000000
A_4	1.000000	1.000000	1.000000

TABLE 1 – Résultats obtenus avec la méthode de Gauss

Pour les quatre matrices la solution obtenue est bien $(1, 1, 1)$. La méthode de Gauss donne donc les résultats attendus.

4.2 Comparaison de Jacobi et Gauss-Seidel

Pour les méthodes itératives on utilise $\epsilon = 10^{-6}$ et un maximum de 1000 itérations.

	Jacobi		Gauss-Seidel	
Matrice	nb itérations	Erreur finale	nb itérations	Erreur finale
A_1	1000	9.66×10^{50}	1000	4.33×10^{199}
A_2	1000	1.85×10^{151}	1000	1.00×10^{301}
A_3	19	3.30×10^{-7}	8	3.08×10^{-7}
A_4	35	6.85×10^{-7}	58	5.47×10^{-7}

TABLE 2 – Comparaison de Jacobi et Gauss-Seidel pour A_1 à A_4

Pour A_1 et A_2 les deux méthodes ne convergent pas. Le nombre maximal de 1000 itérations est atteint et l'erreur devient de plus en plus grande.

Pour A_3 les deux méthodes convergent. Gauss-Seidel est plus rapide avec seulement 8 itérations contre 19 pour Jacobi.

Pour A_4 les deux méthodes convergent aussi, mais ici Jacobi est plus rapide avec 35 itérations contre 58 pour Gauss-Seidel.

4.2.1 Résultats pour la matrice A_5

	Jacobi		Gauss-Seidel	
n	nb itérations	Erreur	nb itérations	Erreur
3	25	6.51×10^{-7}	13	6.51×10^{-7}
6	26	9.01×10^{-7}	14	6.00×10^{-7}
8	26	9.33×10^{-7}	14	6.22×10^{-7}
10	26	9.39×10^{-7}	14	6.26×10^{-7}

TABLE 3 – Résultats pour la matrice A_5

Pour A_5 les deux méthodes convergent pour toutes les tailles testées. Le nombre d'itérations change très peu quand n augmente. Gauss-Seidel demande environ deux fois moins d'itérations que Jacobi.

4.2.2 Résultats pour la matrice A_6

n	Jacobi		Gauss-Seidel	
	nb itérations	Erreur	nb itérations	Erreur
3	36	6.87×10^{-7}	18	5.72×10^{-7}
6	89	8.68×10^{-7}	43	9.68×10^{-7}
8	122	9.66×10^{-7}	59	8.66×10^{-7}
10	150	9.79×10^{-7}	72	9.05×10^{-7}

TABLE 4 – Résultats pour la matrice A_6

Pour A_6 les deux méthodes convergent aussi, mais le nombre d'itérations augmente avec la taille de la matrice. Gauss-Seidel reste plus rapide et demande environ deux fois moins d'itérations que Jacobi.

4.3 Graphiques

Le graphique suivant montre l'évolution de l'erreur pour la matrice A_3.

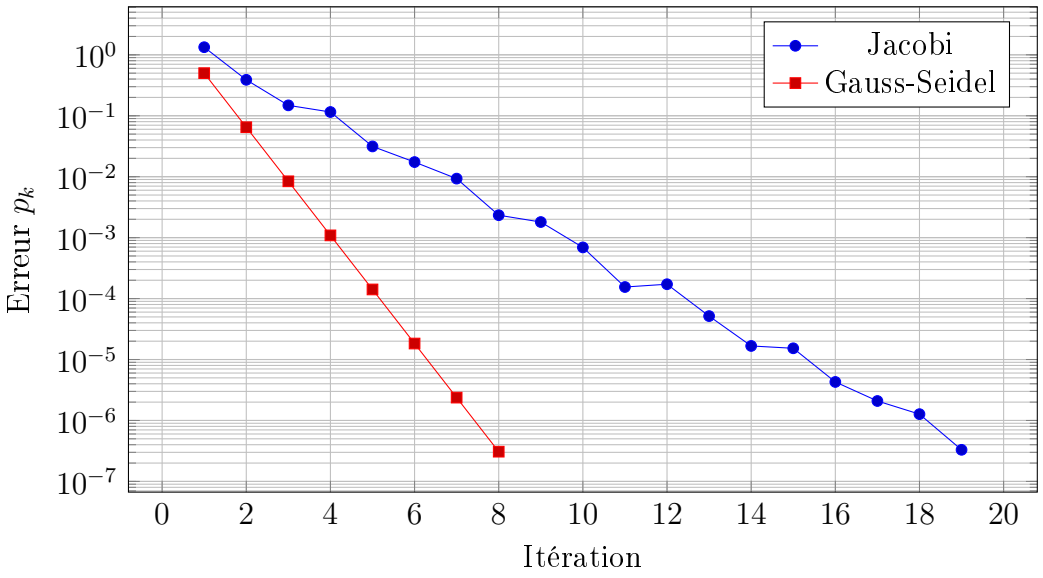


FIGURE 1 – Évolution de l'erreur pour la matrice A_3

On voit que l'erreur diminue beaucoup plus rapidement avec Gauss-Seidel pour cette matrice. La précision demandée est atteinte après 8 itérations contre 19 pour Jacobi.

Le graphique suivant montre l'influence de la taille de la matrice A_6 sur le nombre d'itérations.

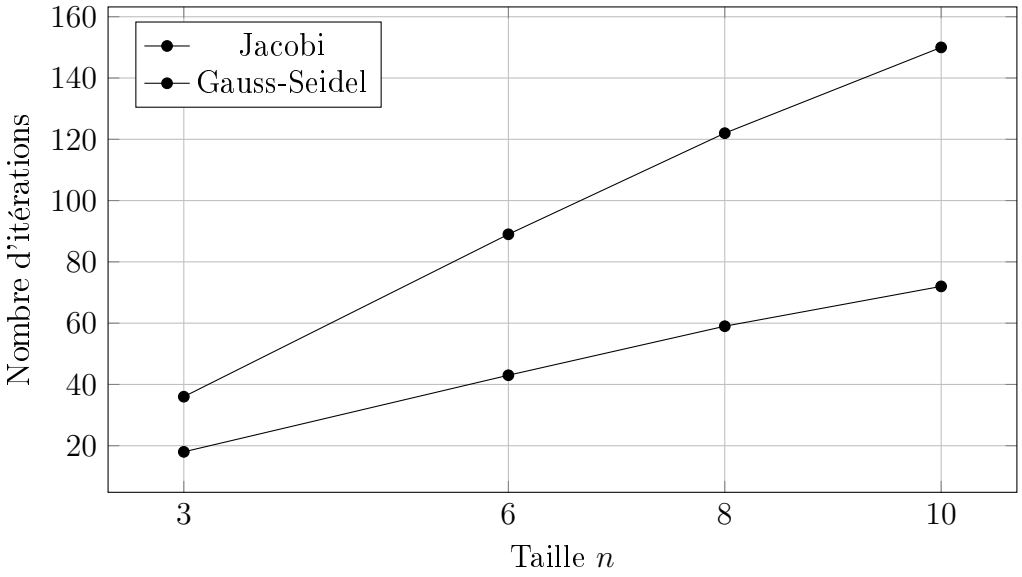


FIGURE 2 – Nombre d'itérations en fonction de n pour A_6

Pour A_6 le nombre d'itérations augmente avec n pour les deux méthodes. On voit aussi que Gauss-Seidel reste plus rapide que Jacobi sur tous les cas testés.

5 Conclusion générale

Les tests montrent que la méthode de Gauss donne la solution attendue pour les quatre matrices testées.

Pour les méthodes itératives la convergence dépend de la matrice. Pour A_1 et A_2, Jacobi et Gauss-Seidel ne convergent pas alors que pour A_3, A_4, A_5 et A_6 les deux méthodes arrivent à la précision demandée.

Gauss-Seidel est généralement plus rapide dans nos tests. C'est le cas pour A_3, A_5 et A_6. Or, ce n'est pas toujours le cas puisque pour A_4 Jacobi atteint la précision demandée avec moins d'itérations.

Les méthodes itératives sont utiles pour des systèmes plus grands mais elles ne convergent pas forcément pour toutes les matrices. La méthode de Gauss donne directement une solution mais la version utilisée ici ne fait pas de choix de pivot et peut donc avoir des problèmes si un pivot est nul ou très petit.

Finalement, le choix de la méthode dépend donc de la matrice et de la taille du système à résoudre.